

Caches

mano@cs.auckland.ac.nz

Caches

- A cache is a small local memory holding items of interest
 - Items of interest include items we used in the recent past and items we may use in the future.
 - Example: a browser cache that holds the web content you looked at recently
- Why is such a cache useful?
 - Avoiding re-fetching items over a (slow) network
 - Processors are a lot faster than memory

Caches

- Assume 5 slots where you can keep numbered (i.e. addressable) items.
 - Items can be anything so long as they have a unique sequence number (i.e. address) with which they can be identified. Example: customer name that is identified through a customer id.
 - We pick items using their ids (e.g. pick a customer using the customer id).
- You want to keep in the 5 slots some items that have been used in the recent past.
 - These 5 slots make up our cache.
- What process should we follow to make our cache work?

Example

Id	Item

Cache

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Example

Id	Item
10	Sue
3	Allan
7	Chris
6	Ben
8	Mitch

Cache

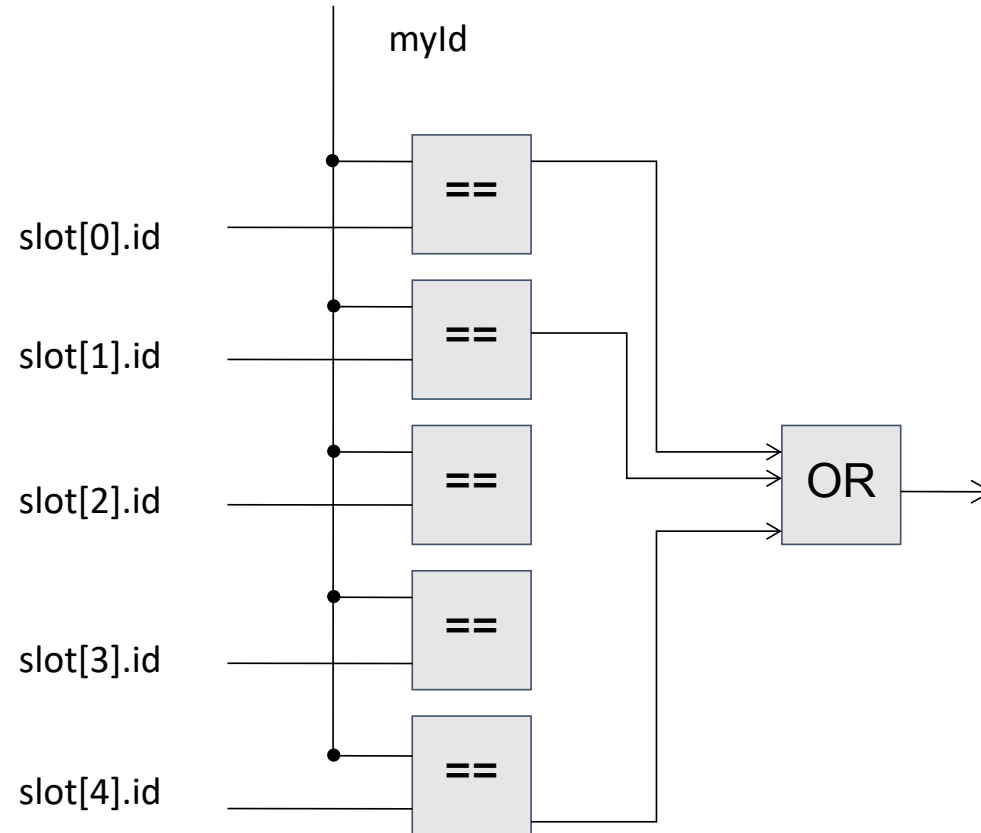
Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Issues with our Cache

- Searching through the cache for a given id is expensive.
 - What is the time complexity of the search, in terms of the size of the cache?
 - How do we do the search in hardware? What is the space complexity?

Hardware Complexity



Issues with our Cache

- Is there a way to reduce the complexity (i.e. speed up the search)?

Direct-Mapped Caches

- In our first attempt, any id from the list can go to any slot in the cache – thus there was a need to search *every* slot in the cache, when we looked for an id.
- Now, we have a variant of the cache where an id can go to *just one* predefined slot in the cache.
 - Id 0 will go to slot 0 (and only slot 0); id 1 will go to slot 1 (and only slot 1); etc;
 - Id 5 will go to slot 0 (and only slot 0); id 6 will go to slot 1 (and only slot 1); etc;
 - This is called a direct-mapped cache. In contrast, our first cache is called a fully-associative cache (where an id can go to any slot).

Direct-Mapped Caches

Id	Item

Cache

Which slot in the cache each id of the data will go to?

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Direct-Mapped Cache

- There is now only one slot where a particular id can be. We don't therefore need to search every slot.
- We improve the complexity.
 - Well, do we?

Direct-Mapped Cache

- There are two problems with our direct-mapped cache.
- What price is this:
 $\text{int where2look} = \text{myId} \% \text{CacheSize}$
- How do we do this mod better?

Direct-mapped Caches

- In hardware, we can replace the mod operation by a mask operation provided that the denominator is a power of 2.
- 11011010 AND 00000011
- Direct-mapped caches in computers therefore have number of entries that is always a power of 2.
 - There are very many other things that are powers of 2 when it comes to computing hardware. Can you think of some others?
 - What is magical about 2?

Direct-mapped Caches

- Suppose we have a direct-mapped cache of size 4 (i.e. with 4 slots).
- Ids 0, 4, 8, 12, ... map to slot 0; ids 1, 5, 9, 13, ... map to slot 1; ids 2, 6, 10, 14, ... map to slot 2; and ids 3, 7, 11, 15, ... map to slot 3.
- Exercise: Imagine the cache has no valid entries to start with (i.e. cold start). Work out if the following accesses are hits or misses in the cache.
 - 1, 2, 3, 4, 2, 3, 1, 4, 5
 - 1, 4, 1, 4, 1, 4, 1, 4, 1
 - 1, 5, 1, 5, 1, 5, 1, 5, 1

0 hit, 5 replace 1.

Direct-Mapped Caches

	Id	Item
0		
1		
2		
3		

Cache

ids 0, 4, 8, 12, ... map to slot 0;
ids 1, 5, 9, 13, ... map to slot 1;
ids 2, 6, 10, 14, ... map to slot 2;
ids 3, 7, 11, 15, ... map to slot 3.

Exercise: Imagine the cache has no valid entries to start with (i.e. cold start). Work out if the following accesses are hits or misses in the cache.

1, 2, 3, 4, 2, 3, 1, 4, 5

1, 4, 1, 4, 1, 4, 1, 4, 1

1, 5, 1, 5, 1, 5, 1, 5, 1

Direct-mapped Caches

- Direct-mapped caches can lead to thrashing: two items go in and out all the time even though there is plenty of space available.
 - A cache miss resulting from thrashing is a conflict miss.
- Is there a way to reduce thrashing?
- Is there a way to get rid of thrashing?
 - Fully-associative cache – our first cache where an id can go to any slot
- Is there a way to reduce thrashing?

抖动 扔掉再找回,

Direct-Mapped Caches

Id	Item

Cache

Which slot in the cache each id of the data will go to?

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Set-Associative Caches

Id	Item	Id	Item

Cache

We have two half-size direct-mapped caches side-by-side.
Which slot in the cache each id of the data will go to?

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Set-Associative Caches

Id	Item	Id	Item

Cache

We have 2 slots where each data id can go to. This reduces thrashing (does it?) but does not prevent it.

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Fully Associative Caches

Id	Item
10	Sue
3	Allan
7	Chris
6	Ben
8	Mitch

Cache

Id	Item
0	Geoff
1	Joe
2	Nora
3	Allan
4	Rick
5	Gary
6	Ben
7	Chris
8	Mitch
9	Dan
10	Sue
11	Fred

List of Items

Fully Associative Caches

- Which of the items to throw out when we need to make room for a new item?
- A replacement policy determines this.
- The most commonly used policy is LRU (least-recently-used): every slot has a counter; every time a slot is accessed, a global counter is incremented and the value stored in the slot counter; the slot with the lowest counter value is chosen for replacement.

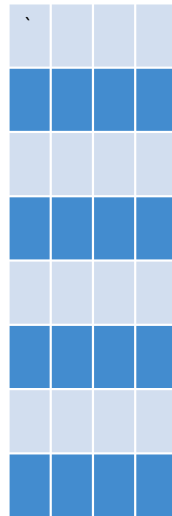
Fully-Associative Caches: Exercise

- Suppose we have a fully-associative cache of size 4 (i.e. with 4 slots).
- Exercise: Imagine the cache has no valid entries to start with (i.e. cold start). Work out if the following accesses are hits or misses in the cache.
 - 1, 2, 3, 4, 2, 3, 1, 4, 5
 - 1, 4, 1, 4, 1, 4, 1, 4, 1
 - 1, 5, 1, 5, 1, 5, 1, 5, 1
 - 2, 5, 1, 3, 4, 2, 5, 1, 3

Further Exercises

- <https://cws.auckland.ac.nz/CW/Core/Coursework?cid=Caches>

Array Layout in Memory



Layout is row by row.



Two ways to sum a 2-D array

```
const int MAXX = 5;
const int MAXY = 4;
int array[MAXX][MAXY]; // init
int sum = 0;
for ( int i = 0; i < MAXX; ++i )
{
    for ( int j = 0; j < MAXY; ++j )
    {
        sum += array[i][j];
    }
}
```

```
const int MAXX = 5;
const int MAXY = 4;
int array[MAXX][MAXY]; // init
int sum = 0;
for ( int j = 0; j < MAXY; ++j )
{
    for ( int i = 0; i < MAXX; ++i )
    {
        sum += array[i][j];
    }
}
```

Two ways to sum a 2-D array

```
const int MAXX = 5;
const int MAXY = 4;
int array[MAXX][MAXY]; // init
int sum = 0;
for ( int i = 0; i < MAXX; ++i )
{
    for ( int j = 0; j < MAXY; ++j )
    {
        sum += array[i][j];
    }
}
```

a(0,0), a(0,1), a(0,2), a(0,3),
a(1,0), a(1,1), a(1,2), a(1,3),,
a(2,0), a(2,1), ...

Access order = memory layout order

faster

```
const int MAXX = 5;
const int MAXY = 4;
int array[MAXX][MAXY]; // init
int sum = 0;
for ( int j = 0; j < MAXY; ++j )
{
    for ( int i = 0; i < MAXX; ++i )
    {
        sum += array[i][j];
    }
}
```

a(0,0), a(1,0), a(2,0), a(3,0), a(4,0),
a(0,1), a(1,1), a(2,1), a(3,1), a(4,1),
a(0,2), a(1,2), ...

Access order != memory layout order

Access Sequence #1

空间局限性

no one
bring items
around it

Array			
a(0,0)	a(0,1)	a(0,2)	a(0,3)
a(1,0)	a(1,1)	a(1,2)	a(1,3)
a(2,0)	a(2,1)	a(2,2)	a(2,3)
a(3,0)	a(3,1)	a(3,2)	a(3,3)
a(4,0)	a(4,1)	a(4,2)	a(4,3)
a(9,0)	a(9,1)	a(9,2)	a(9,3)

Memory

Cache			

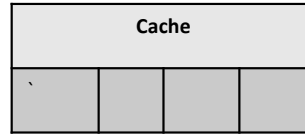
a(0,0)
a(0,1)
a(0,2)
a(0,3)
a(1,0)
a(1,1)
a(1,2)
a(1,3)
a(2,0)
a(2,1)
a(2,2)
a(2,3)
a(3,0)

miss hit
take → a(0,0), a(0,1), a(0,2), a(0,3),
a(1,0), a(1,1), a(1,2), a(1,3),
a(2,0), a(2,1) ...
Access order = memory layout order

Work out the cache hits/misses for the memory accesses.

Access Sequence #2

Array			
a(0,0)	a(0,1)	a(0,2)	a(0,3)
a(1,0)	a(1,1)	a(1,2)	a(1,3)
a(2,0)	a(2,1)	a(2,2)	a(2,3)
a(3,0)	a(3,1)	a(3,2)	a(3,3)
a(4,0)	a(4,1)	a(4,2)	a(4,3)
a(9,0)	a(9,1)	a(9,2)	a(9,3)



Memory

a(0,0)
a(0,1)
a(0,2)
a(0,3)
a(1,0)
a(1,1)
a(1,2)
a(1,3)
a(2,0)
a(2,1)
a(2,2)
a(2,3)
a(3,0)

a(0,0), a(1,0), a(2,0), a(3,0), a(4,0),
a(0,1), a(1,1), a(2,1), a(3,1), a(4,1),
a(0,2), a(1,2), ...

Access order != memory layout order

Work out the cache hits/misses for the memory accesses.

¿?